

Roni Butbul · Shawn McDonald · Thomas Cahill · Raymond Pridgen

Heat Diffusion Simulation: Parallel Performance Analysis

Scalability of Serial, Pthreads, OpenMP,
MPI, and Hybrid Implementations



Project Purpose

Main goal:

Analyze how different parallel programming models perform on the same 2D heat diffusion simulation.

What we compared:

- Serial baseline
- Pthreads
- OpenMP
- MPI
- Hybrid MPI + OpenMP

Main question:

Which model performs best, and where does overhead begin to limit scalability?



Simulation Overview

This project simulates heat spreading across a 2D grid.

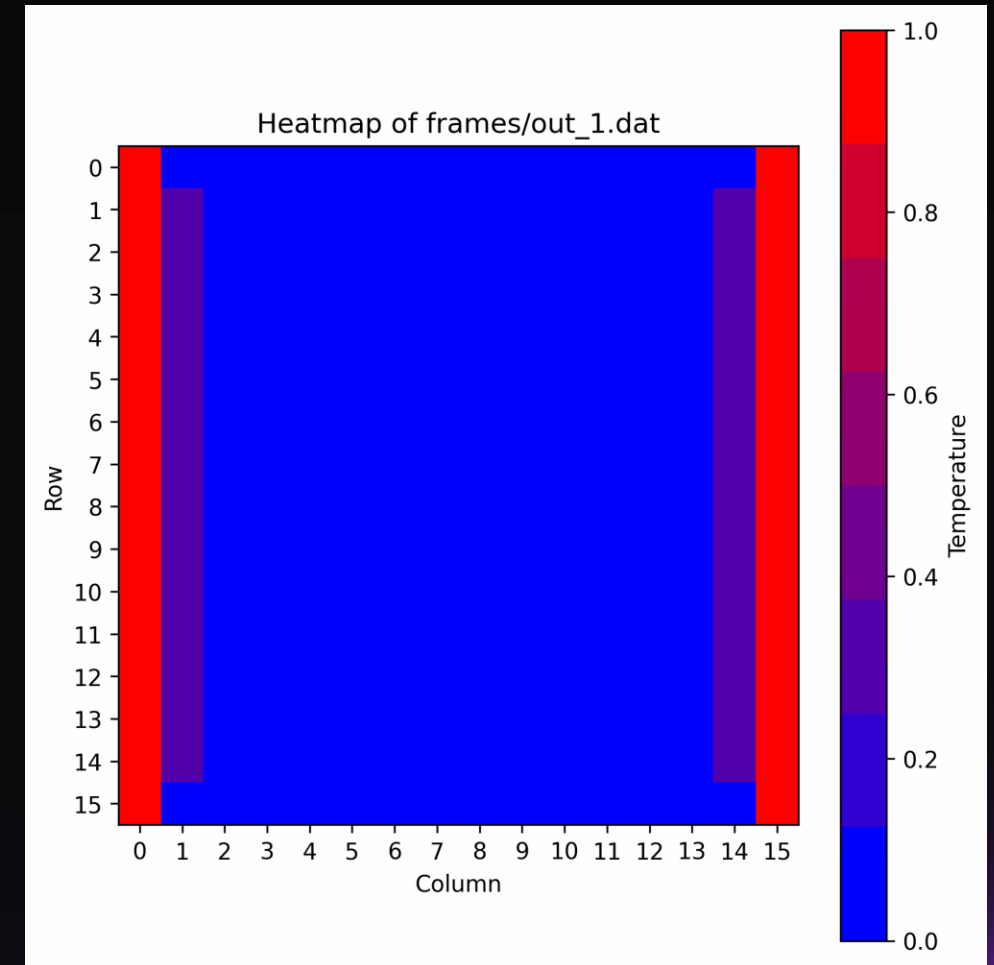
Boundary conditions:

Left and right edges stay hot at **1.0**

Top and bottom edges stay cool at **0.0**

Interior cells begin at **0.0**

Over time, heat spreads inward from the hot side boundaries while the top and bottom remain fixed cooling boundaries.



The 9-Point Stencil

Each interior cell is updated using:

The cell itself

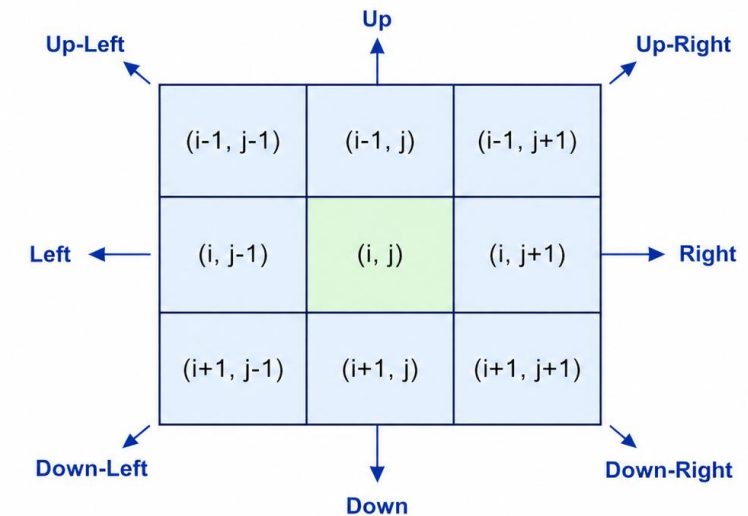
The 8 surrounding neighbors

Average of all 9 values

Why this matters:

The stencil lets heat spread in all directions, including diagonally.

$$new[i, j] = \frac{1}{9} \sum_{d_i=-1}^1 \sum_{d_j=-1}^1 old[i + d_i, j + d_j]$$



Why Parallelization is needed?

The workload grows as: $O(N^2 \cdot t)$

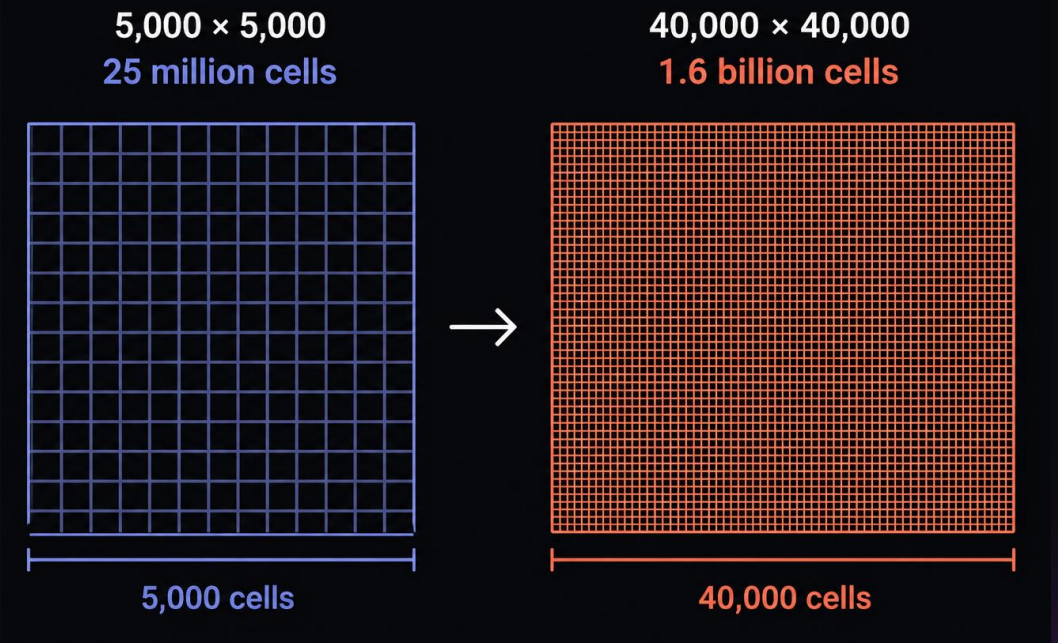
So increasing the grid size has a huge effect.

Example from our project:

5,000 × 5,000 grid = **25 million cells**

40,000 × 40,000 grid = **1.6 billion cells**

This is why the serial version becomes expensive and why parallelism is necessary for larger simulations.

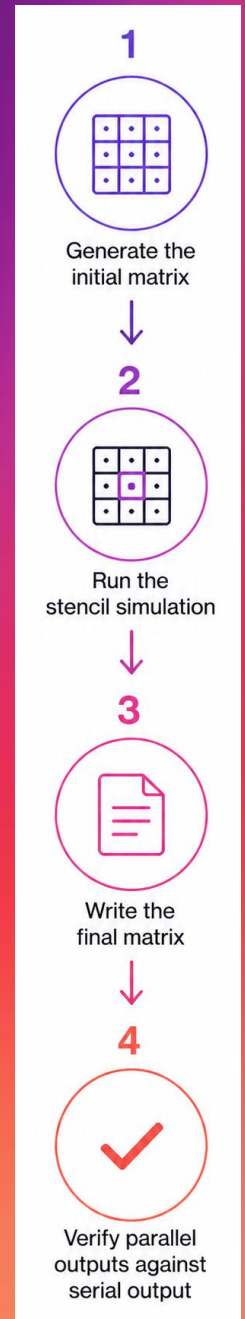


Program Design

Our workflow followed four main steps:

1. Generate the initial matrix
2. Run the stencil simulation
3. Write the final matrix
4. Verify parallel outputs against serial output

The same binary file format was used across all versions so every implementation could be tested consistently.



Parallel Models – Shared Memory

All threads work on the same grid and must synchronize after each iteration.

Pthreads

- Manually divides rows among threads
- Uses custom barriers
- Gives more control, but more responsibility

OpenMP

- Uses compiler directives
- Shorter and simpler code
- Runtime handles thread scheduling



Parallel Models - Distributed Memory & Hybrid

Work is divided across processes with separate memory and requires communication between them.

MPI

- Splits the grid into horizontal row blocks
- Each rank owns part of the grid
- Neighboring ranks exchange ghost rows each iteration

Hybrid

- MPI divides the grid across ranks
 - OpenMP parallelizes work inside each rank
- *The hybrid version was expected to combine the benefits of both models, but our results showed that the added complexity did not outperform pure MPI.



Experimental Setup

Grid sizes:

5k

10k

20k

40k

Parallel Counts (p):

1

2

4

8

16

Outputs Measured:

Overall Time

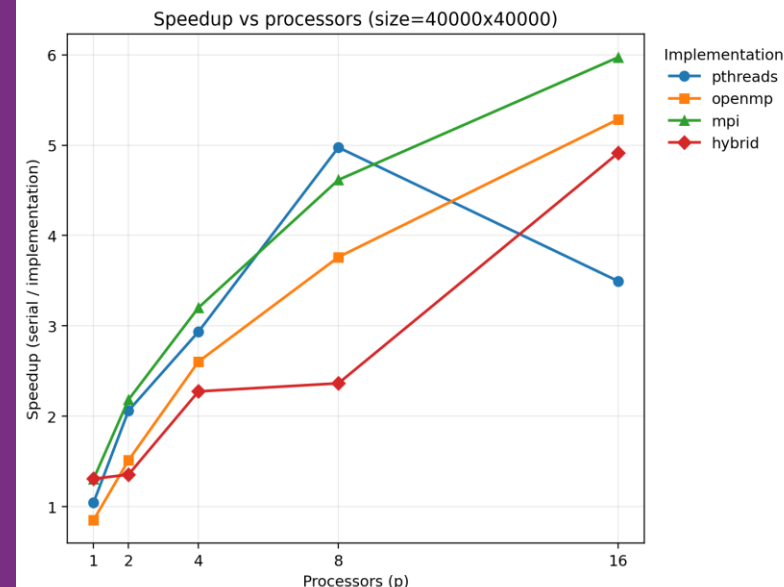
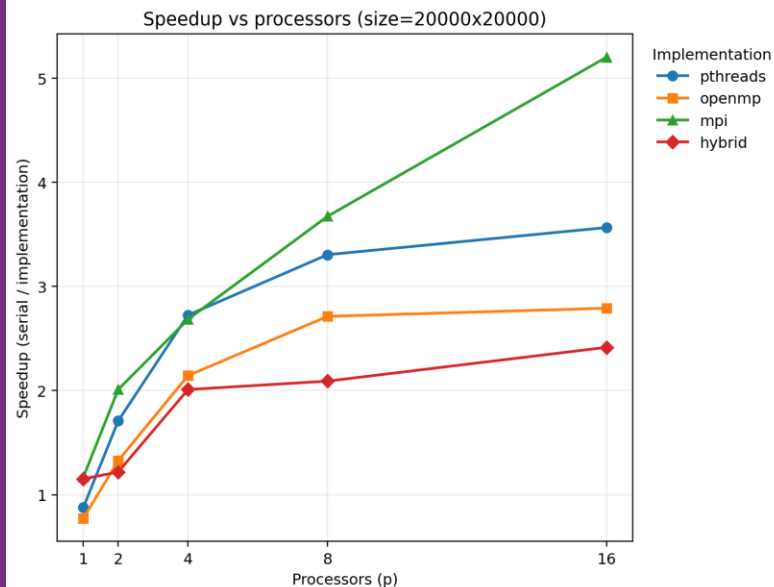
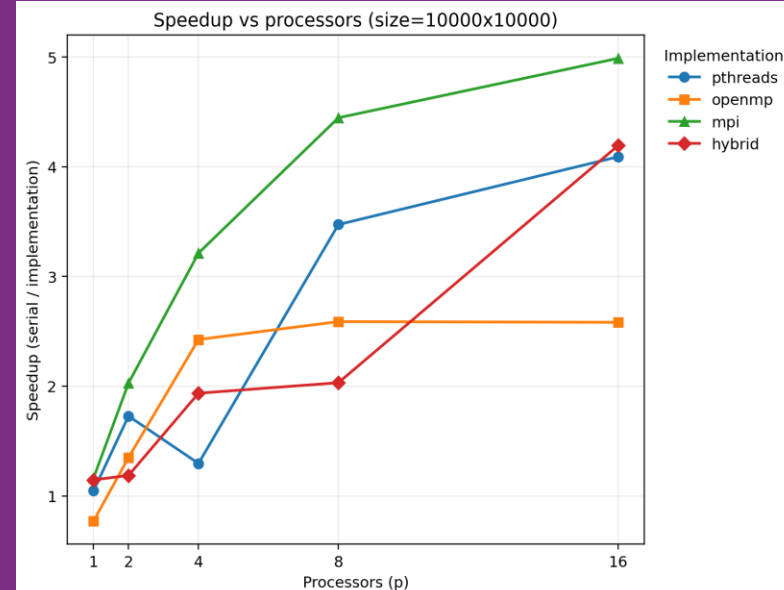
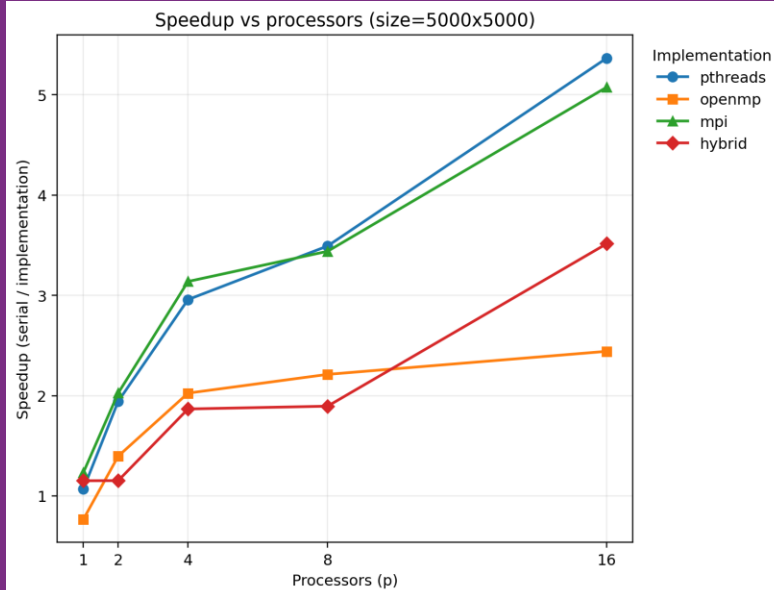
Computational Time

Other = Overall - Compute

Iteration: t=13



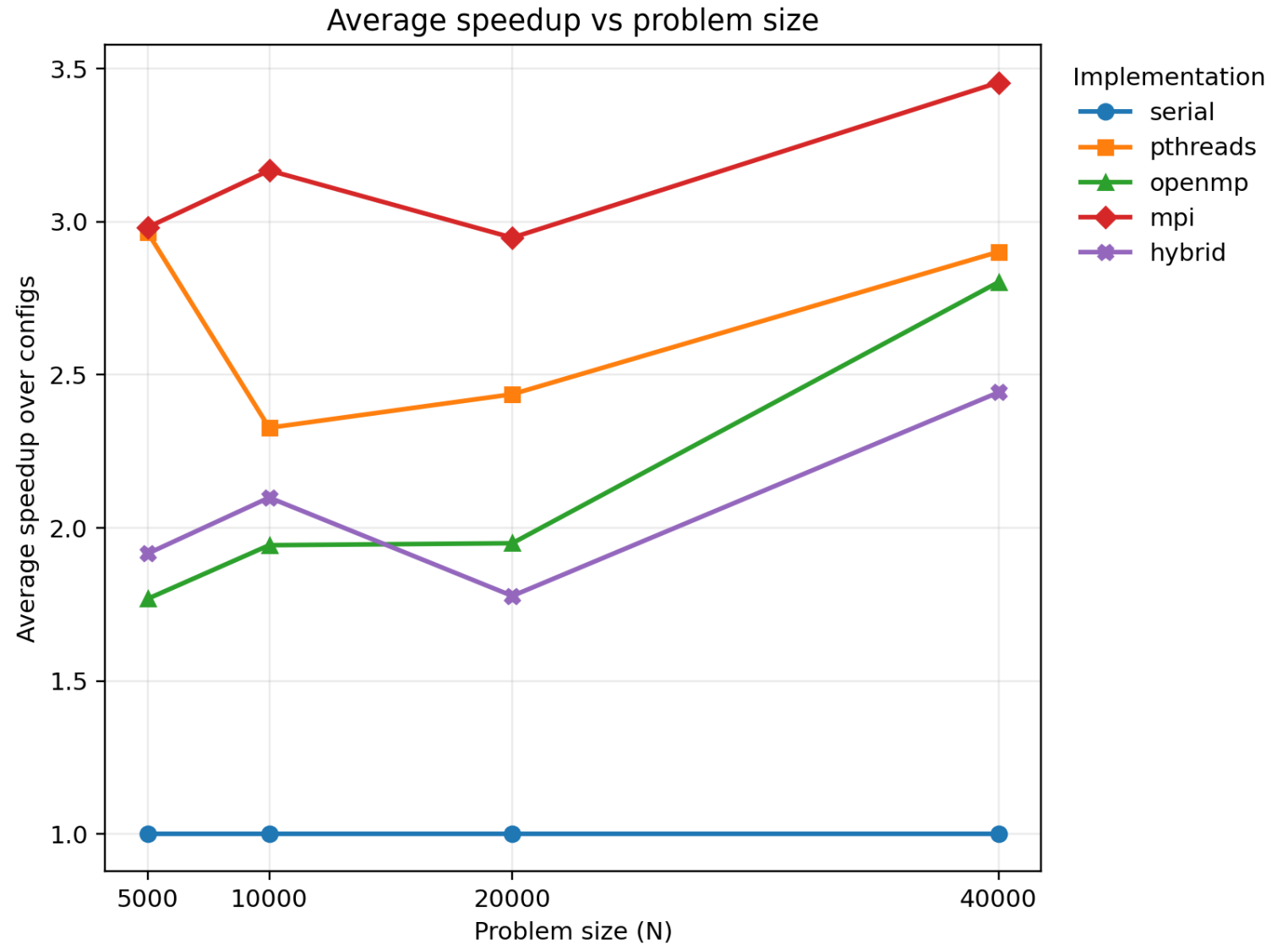
Speedup represented as an increase over the serial baseline



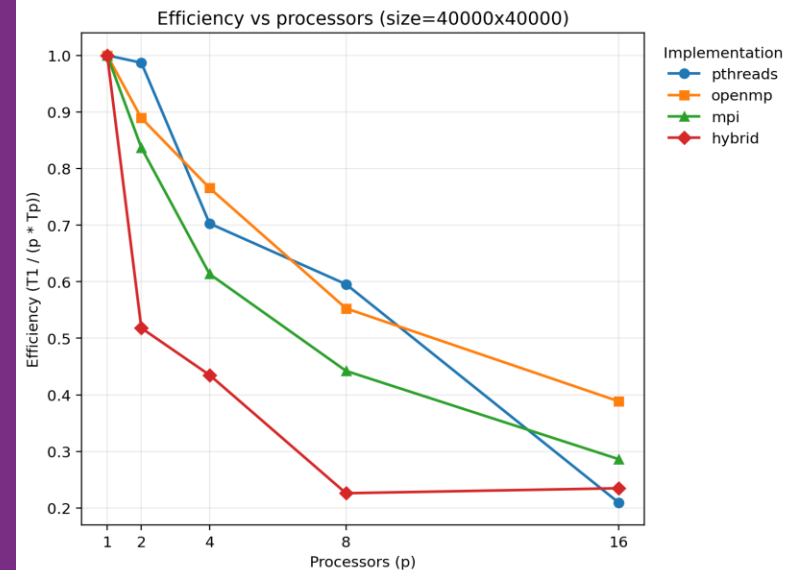
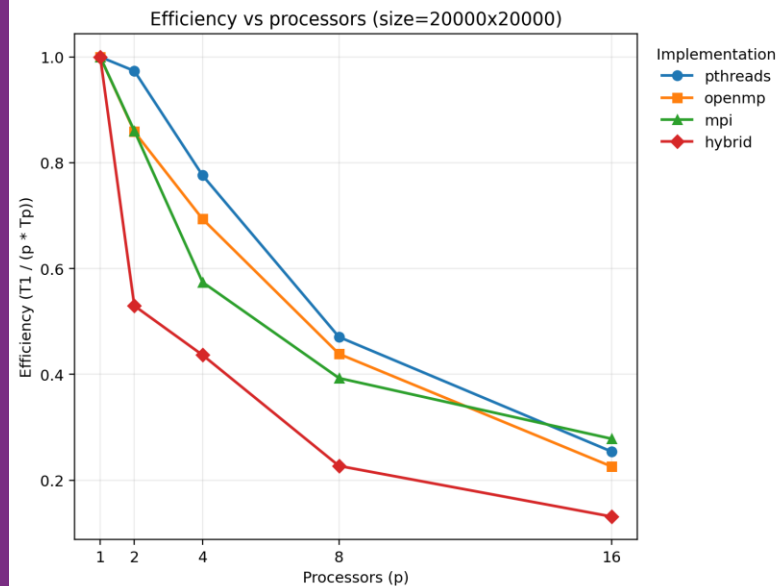
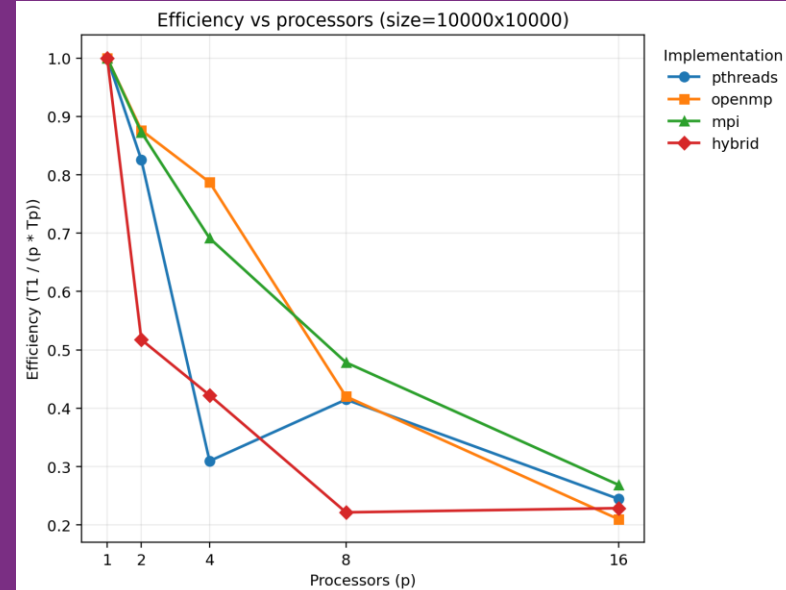
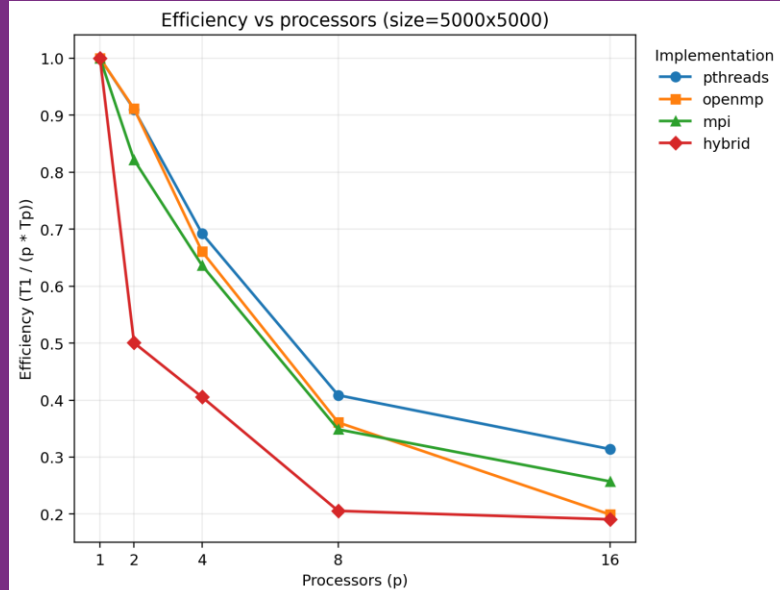
Average Speedup Over Problem Size

As the problem grows:

- MPI consistently outperforms other implementations
- Pthreads can rival MPI but is sensitive to uneven loads
- OpenMP has the most consistent speedup increase with processors
- Hybrid performs the worst



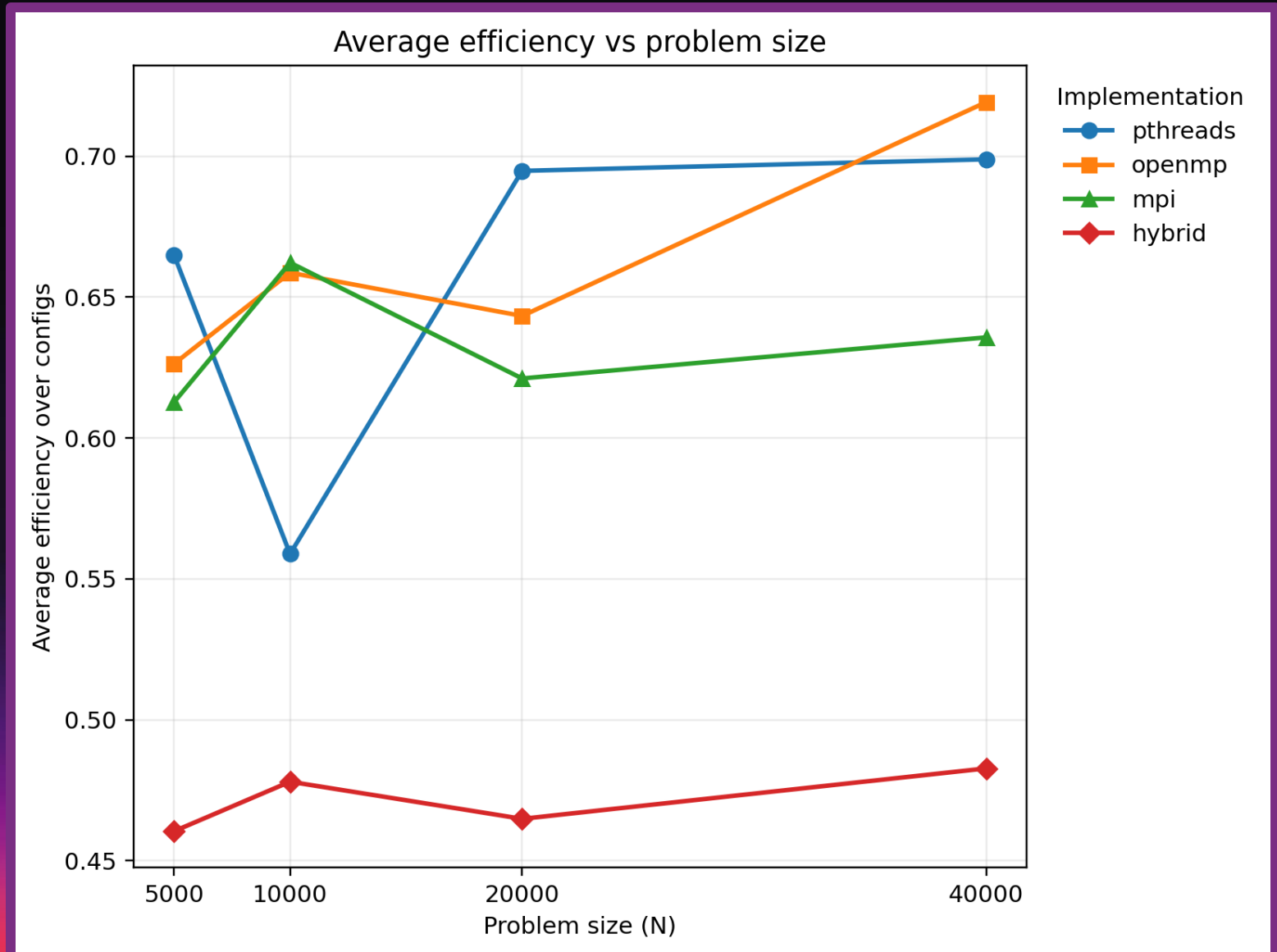
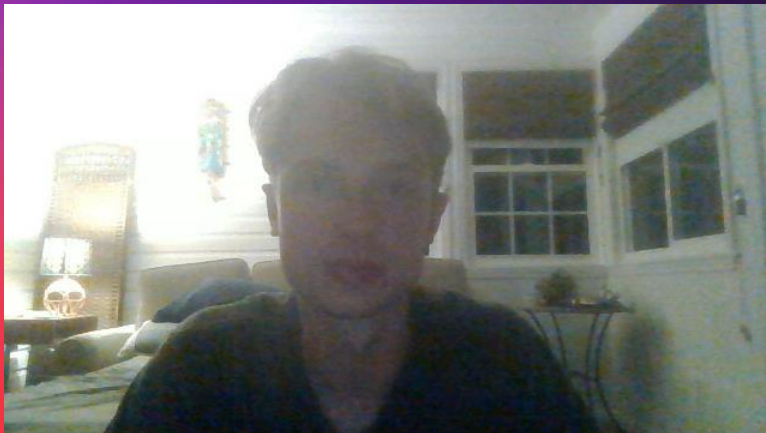
Efficiency across implementations and processors



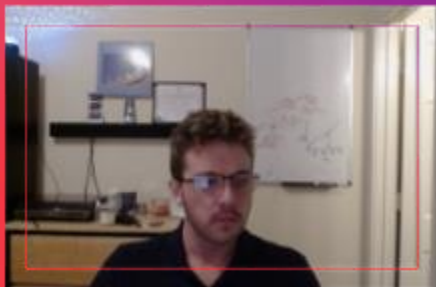
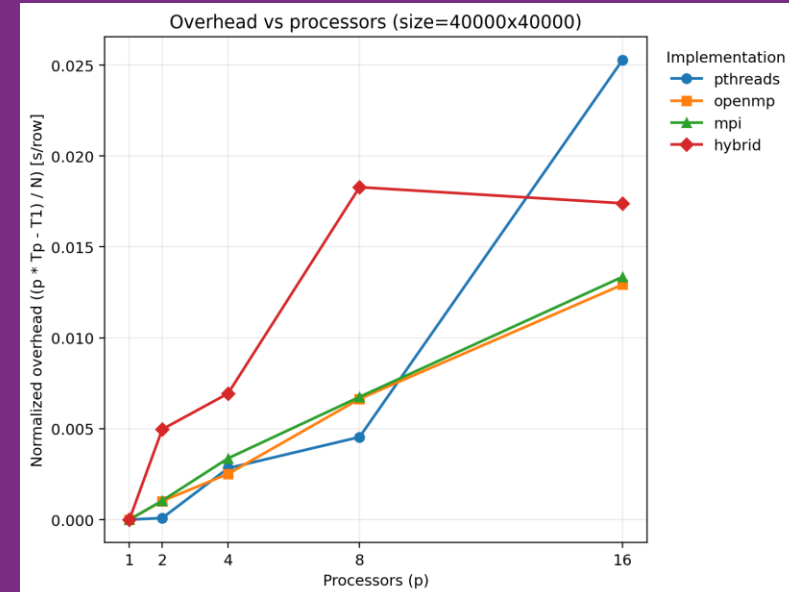
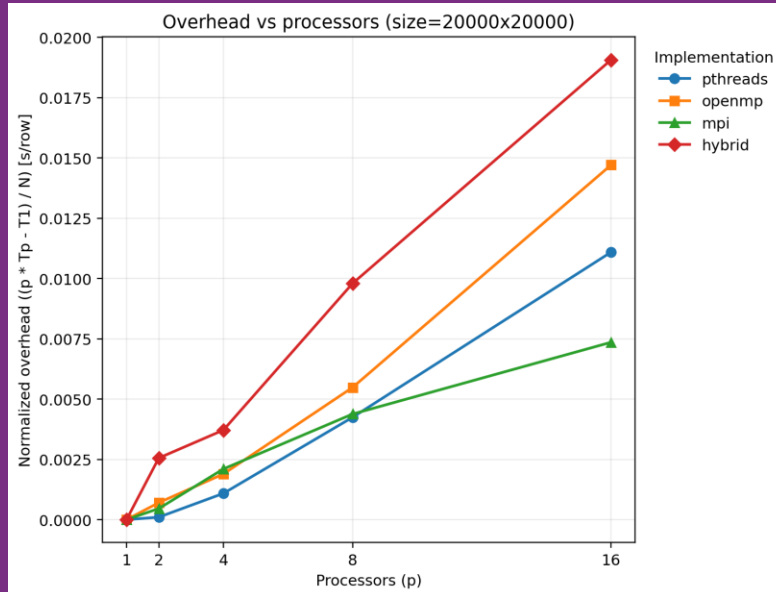
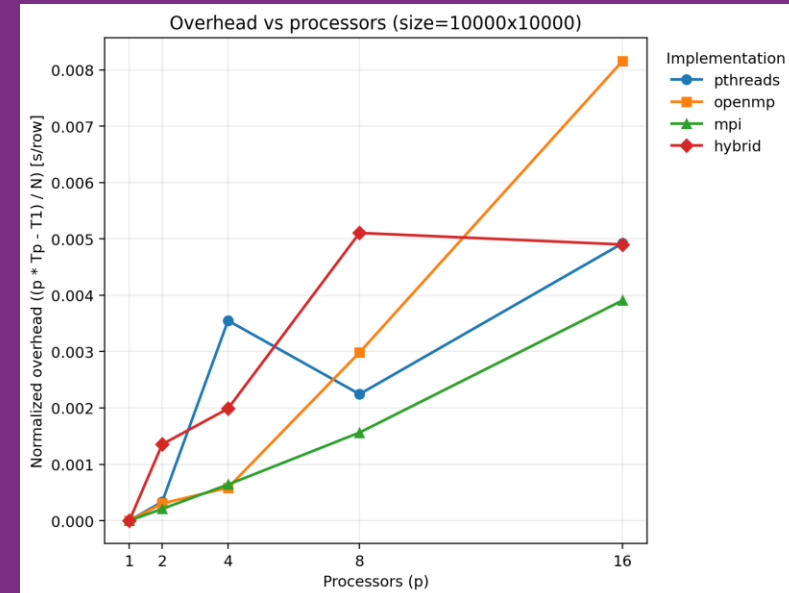
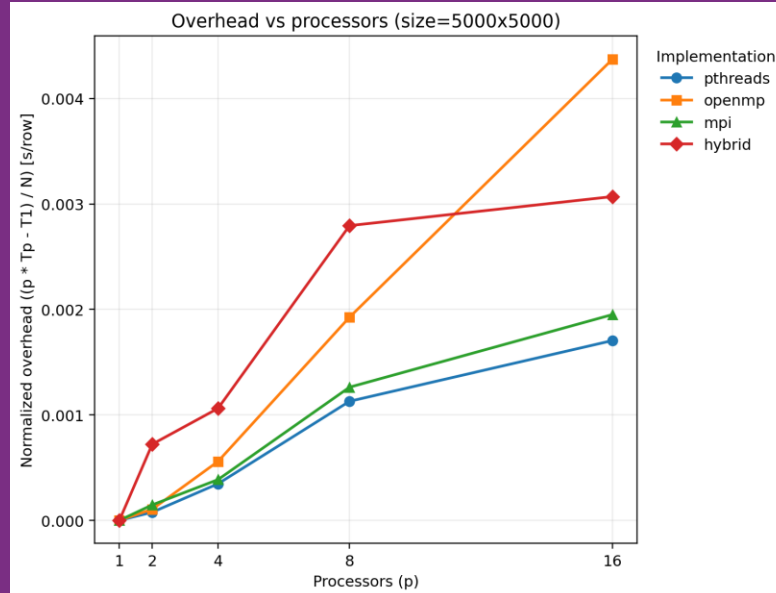
Average efficiency over problem size

As the problem grows:

- Hybrid approach is consistently the least efficient at distributing work
- Pthreads and OpenMP are the most efficient, excluding outliers



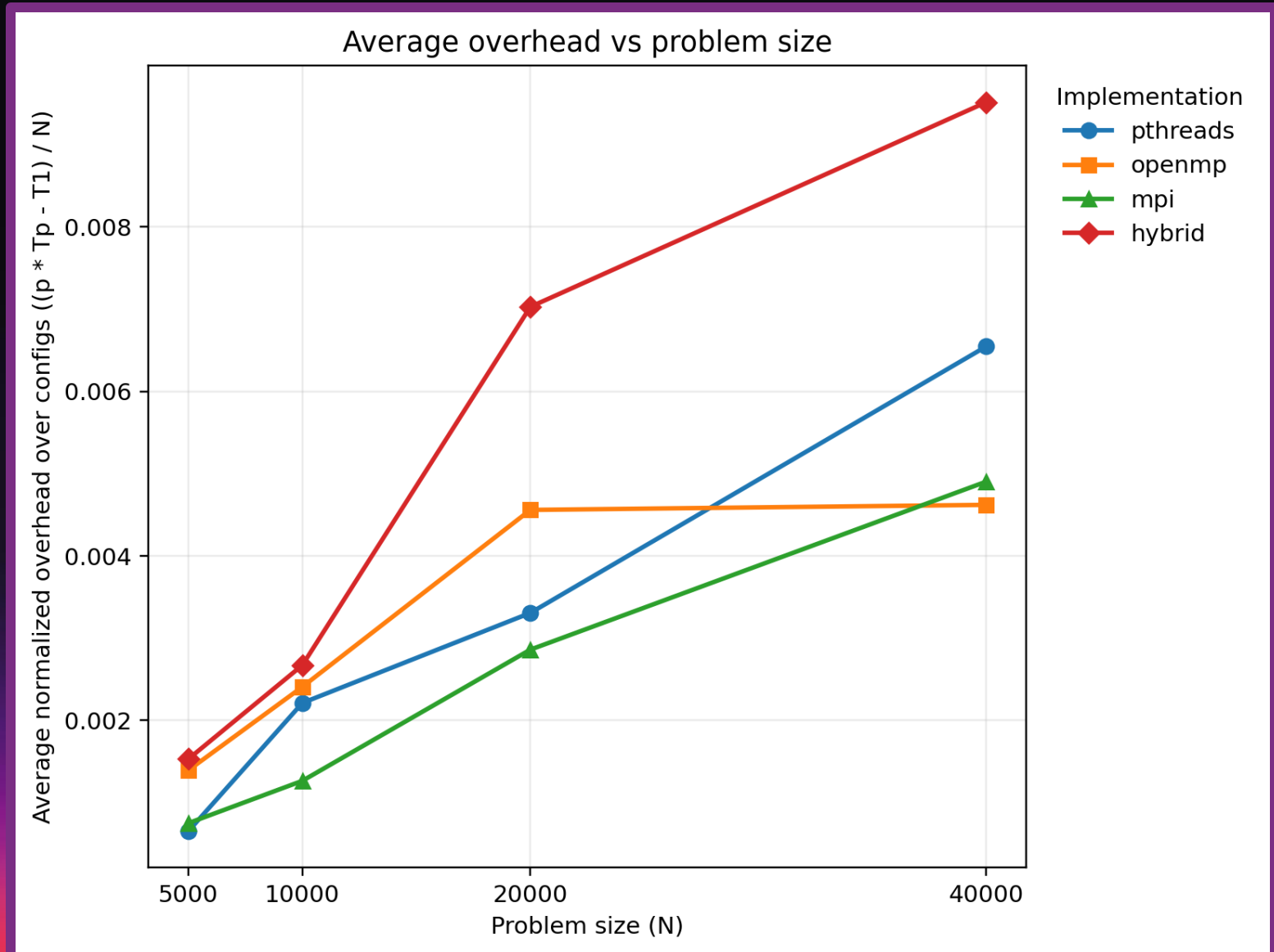
Overhead across implementations and processors



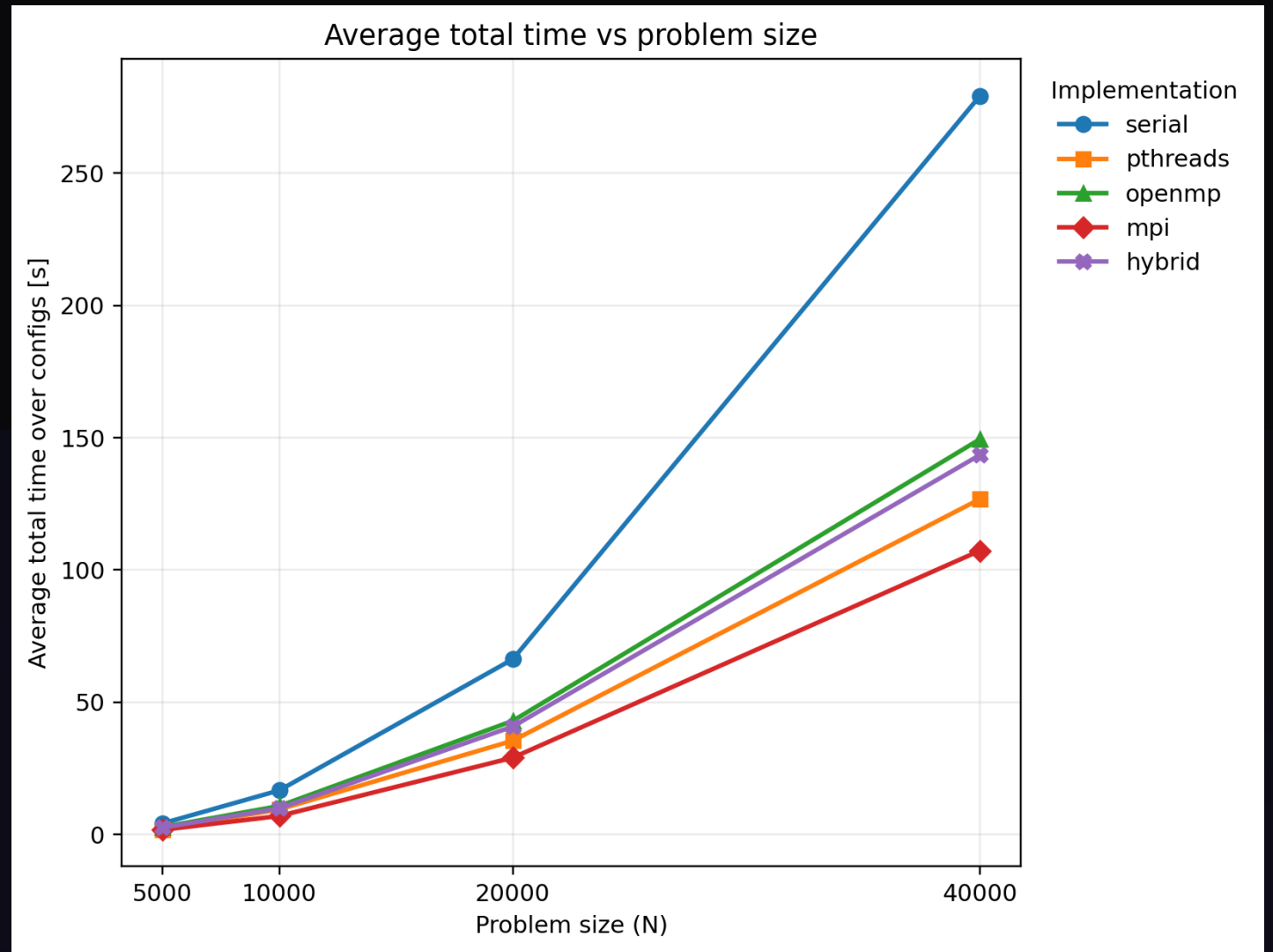
Average overhead over problem size

As the problem grows:

- Hybrid approach encounters the most overhead
- MPI generally has the least overhead
- MPI achieves higher speedup with lower overhead compared to other implementations
- OpenMP overhead grows linearly from 20,000 to 40,000



Overall Runtime



Conclusion

- Parallelism significantly improves performance
- MPI (in this case) was the best for large-scale problems
- Overhead limits scalability
- Optimal solution depends on:
 - problem size
 - process count
 - system architecture

